

Skin Sense

Project Team

Ayan Asif	22I-1097
Usman Haroon	22I-1177
Ibrahim Azhar	22I-0928

Session 2022–2026

Supervised by

Ms. Azka Atiq

Co-Supervised by

Dr. Ahmad Raza



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

May, 2026

Contents

1	Introduction	6
1.1	Problem Statement	7
1.2	Scope	7
1.3	Modules	8
1.3.1	Client Web App: Patient Portal	8
1.3.2	Clinician Web App: Doctor Portal	8
1.3.3	AI Inference Module	8
1.3.4	Speech and NLP Module	9
1.3.5	Security Module	9
1.4	Validity and Effectiveness of Proposed Solution	9
1.4.1	Competitive Analysis	9
1.4.2	Justification for PWA over Native Mobile App	10
1.5	User Classes and Characteristics	10
2	Project Requirements	11
2.1	Use Cases and Use Case Diagram	11
2.1.1	Detailed Use Cases	12
2.2	Functional Requirements	15
2.2.1	Module 1: Patient Portal	15
2.2.2	Module 2: Healthcare Worker Portal	15
2.2.3	Module 3: AI Inference Module	16
2.2.4	Module 4: Speech and NLP Module	16
2.3	Non-Functional Requirements	17
2.3.1	Performance Requirements	17
2.3.2	Reliability Requirements	17
2.3.3	Usability Requirements	18
2.3.4	Security Requirements	18
3	System Overview	19
3.1	Architectural Design	19
3.1.1	Architecture Selection and Justification	19
3.1.2	Architectural Diagram	20
3.1.3	Layer Descriptions	21
3.2	Design Models	23
3.2.1	Sequence Diagrams	23
3.2.2	Activity Diagram	25

3.2.3	Class Diagram	26
3.2.4	Error Handling and Alternative Flows	27
3.3	Data Design	28
3.3.1	Entity-Relationship Diagram	28
3.3.2	Entity Relationships and Cardinalities	29
3.3.3	Database Indexes	29
4	ML Pipeline	30
4.1	Overview	30
4.2	Dataset Curation	30
4.2.1	Selection Criteria	30
4.2.2	Datasets Used	31
4.2.3	Final Dataset Composition	31
4.2.4	Train / Validation / Test Split	31
4.3	Model Training Iterations	32
4.3.1	Baseline: MobileNetV3-Small	32
4.3.2	Teacher Model: EfficientNet-B3	32
4.3.3	Knowledge Distillation	33
4.3.4	Final Model: MobileNetV3-Large (Distilled)	34
4.4	Model Comparison Summary	34
4.5	Key Findings	34
5	Testing and Quality Assurance	36
5.1	Testing Overview	36
5.2	Testing Infrastructure	36
5.2.1	Technology Stack	36
5.2.2	Test Environment Setup	37
5.2.3	Mocking Strategy	37
5.3	Test Coverage by Service	38
5.4	Backend (Server) Tests	39
5.4.1	Authentication Tests (auth.test.js)	39
5.4.2	Screening Pipeline Tests (screening.test.js)	41
5.4.3	Middleware Tests (middleware.test.js)	42
5.5	Frontend (Client) Tests	43
5.5.1	Screening Hook Tests (useScreening.test.jsx)	43
5.5.2	Voice Recorder Hook Tests (useVoiceRecorder.test.jsx)	44
5.5.3	Image Validation Tests (imageValidation.test.js)	45
5.6	Inference (Python) Tests	45
5.6.1	Prediction and Health Tests (test_predict.py)	46

5.6.2	Transcription Tests (test_transcribe.py)	46
5.6.3	Heatmap Endpoint Tests (test_heatmap.py)	46
5.6.4	Utility Function Tests (test_utils.py)	47
5.7	Key Design Decisions in the Test Suite	48
5.7.1	Graceful Fallback Tested via Natural Failure	48
5.7.2	Random-Weight Inference Testing	49
5.7.3	Path Traversal Security	49
5.7.4	Selective Fake Timer Scope	49
5.7.5	Two-Layer Validation	49
5.8	Incorrect Examples Handled	50
5.9	Edge Cases Handled	50
5.10	ML Evaluation	51

List of Figures

2.1	SkinSense Use Case Diagram	11
3.1	SkinSense Five-Layer Logical Architecture	20
3.2	SkinSense Deployment Modes: Online vs Offline	21
3.3	System-Level Sequence Diagram: Patient Screening Workflow	23
3.4	System-Level Sequence Diagram: Clinician Feedback Workflow	24
3.5	Activity Diagram: Complete Patient Screening Workflow with Swimlanes	25
3.6	Class Diagram for SkinSense System	26
3.7	Entity-Relationship Diagram for SkinSense Database	28

List of Tables

1.1	Competitive Analysis: SkinSense vs. Existing Solutions	9
1.2	User Classes and Characteristics	10
2.1	Performance Requirements for SkinSense	17
2.2	Reliability Requirements for SkinSense	17
2.3	Usability Requirements for SkinSense	18
2.4	Security Requirements for SkinSense	18
3.1	Error Handling in Screening Workflow	27
3.2	Entity Relationships and Cardinalities	29
3.3	Database Indexes for Performance	29
4.1	Dataset Sources and Contributions	31
4.2	Final Dataset Composition by Class	31
4.3	Baseline MobileNetV3-Small Per-Class Test Results	32
4.4	EfficientNet-B3 Teacher Model Per-Class Test Results	33
4.5	MobileNetV3-Large Distilled Student Per-Class Test Results	34
4.6	Full Model Comparison	34
5.1	Testing Frameworks by Service	36
5.2	Test Coverage Summary by File	39
5.3	Authentication Endpoint Test Distribution	40
5.4	Selected Authentication Test Cases	40
5.5	Screening Pipeline Test Distribution	41
5.6	Middleware Test Distribution	42
5.7	Screening Hook Test Distribution	43
5.8	Voice Recorder Hook Test Distribution	44
5.9	Image Validation Test Distribution	45
5.10	Numeric and Shape Contracts Verified for POST /predict	46
5.11	Heatmap Endpoint Test Distribution	47
5.12	Utility Function Test Distribution	48

Chapter 1

Introduction

Skin diseases affect an estimated 1.9 billion people worldwide at any given time, yet dermatological care remains one of the least accessible medical specialties. The disparity is most acute in low and middle income countries: Pakistan has fewer than 1,500 registered dermatologists for a population exceeding 230 million, with the overwhelming majority concentrated in major urban centres. Rural patients have no viable pathway to specialist evaluation without travelling distances that are prohibitive in cost and time. Infrastructure constraints compound the problem. The Pakistan Telecommunication Authority reports that 68% of rural areas in Punjab and KPK experience intermittent connectivity with average daily downtimes of 4 to 6 hours [1], average mobile speeds of 2 to 5 Mbps fall well below the threshold for real-time cloud inference, and the Pakistan Bureau of Statistics 2023 data places rural literacy below 40% [2], excluding text-heavy interfaces from a majority of intended users.

Existing AI dermatology solutions including SkinVision, FirstDerm, Skinive, and DermEngine fail to address these realities. All four require continuous internet connectivity, charge \$5 to \$50 per screening, offer no Urdu or voice-based interaction, and are trained predominantly on lighter skin tones, introducing systematic bias against South Asian patients. SkinSense is designed to close this gap. It is an AI-powered dermatological screening system built specifically for rural and low-resource healthcare settings in Pakistan. Patients and healthcare workers can capture or upload an image of a skin condition and receive a probabilistic screening result across seven common dermatological conditions: Vitiligo, Melasma, Psoriasis, Eczema, Tinea, Contact Dermatitis, and Seborrheic Dermatitis. The system runs inference locally after an initial model download, supports voice-based symptom input in Urdu and English via a fine-tuned Whisper ASR model, provides GradCAM heatmaps for visual explainability, and includes a human-in-the-loop clinician feedback mechanism that positions it as a clinical decision support tool rather than an autonomous diagnostic system.

This report is structured as follows. Chapter 2 presents the project requirements including use cases, functional requirements, and non-functional requirements. Chapter 3 provides a comprehensive system overview covering the layered architectural design with justification, sequence and activity diagrams, class diagram, and database design. Chapter 4 details the ML pipeline. Chapter 5 presents the quality assurance and testing outcomes.

1.1 Problem Statement

Timely and accurate dermatological screening in rural Pakistan is currently unachievable for the majority of the population due to three compounding barriers: a critical shortage of dermatology specialists outside urban centres, the unreliability of existing cloud-dependent AI screening solutions under real-world connectivity and hardware constraints, and the absence of culturally and linguistically appropriate digital health tools for South Asian, low-literacy users.

Pakistan has fewer than 1,500 registered dermatologists for a population exceeding 230 million, with the overwhelming majority practising in major urban centres. Rural patients experiencing skin conditions have no viable pathway to specialist evaluation without travelling distances that are prohibitive in terms of cost and time. Community health workers and rural clinic staff, who represent the first point of contact for most of these patients, lack the tools and training to perform reliable preliminary screening.

Existing AI-powered dermatology solutions do not solve this problem. They require stable broadband connectivity that does not exist in 68% of rural Punjab and KPK, operate on subscription pricing that is economically inaccessible to low-income populations, provide no support for Urdu, and are trained on datasets that systematically underrepresent darker skin tones. No currently available solution operates offline, provides visual explainability for clinical trust, or supports voice-based interaction for users with limited literacy.

This creates a clearly defined, evidence-backed need: an on-device, explainable, multilingual dermatological screening system that functions under intermittent connectivity, costs less than Rs. 1 per screening, and is designed from first principles for the infrastructure and demographic realities of rural Pakistan. SkinSense is designed to fulfil this need.

1.2 Scope

The scope of the SkinSense project includes the following:

- Development of an AI-assisted system for preliminary screening of seven common dermatological conditions.
- Use of dermatological image analysis as the primary screening input modality.
- Support for optional voice and text symptom input.
- Presentation of screening results as probabilistic confidence scores rather than definitive diagnoses.

- Generation of GradCAM visual explainability heatmaps.
- Patient-facing and clinician-facing interfaces with role-based access control.
- Human-in-the-loop workflow for clinician review and feedback.
- Offline-tolerant deployment on devices with minimum 2GB RAM.
- Evaluation using regionally relevant datasets including South Asian skin tones.
- Secure authentication, session management, and encrypted data storage.
- Multilingual UI supporting Urdu and English.

1.3 Modules

1.3.1 Client Web App: Patient Portal

1. Role selection and patient entry flow.
2. Image capture or upload of affected skin region.
3. Symptom input via voice (Urdu/English) and optional text.
4. Screening result view: probabilistic confidence scores across all seven conditions.
5. GradCAM explainability heatmap overlay.
6. Multilingual UI (Urdu and English).
7. Nearest clinic guidance via Map API based on user location.

1.3.2 Clinician Web App: Doctor Portal

1. Clinician authentication and role-based access.
2. Detailed screening output view with full confidence distribution.
3. GradCAM heatmaps and comparative viewing.
4. Case review history and patient record linkage.
5. Human-in-the-loop feedback: accept, dispute, or correct screening output.
6. Audit logging of all clinician actions.

1.3.3 AI Inference Module

1. Dataset preprocessing, augmentation, and stratified train/val/test splits.

2. Knowledge distillation: EfficientNet-B3 teacher to MobileNetV3-Large student.
3. Edge-optimised inference achieving 71% test accuracy at 3.4M parameters.
4. Probabilistic confidence outputs across seven conditions.
5. GradCAM heatmap generation for all inference results.

1.3.4 Speech and NLP Module

1. Custom Whisper v3 Turbo Large ASR for Urdu and English speech-to-text.
2. Symptom keyword extraction via NLP.
3. Audio fallback: automatic raw blob transmission if WAV conversion fails.

1.3.5 Security Module

1. JWT-based authentication with bcrypt password hashing (cost factor ≥ 12).
2. RBAC separating Patient and Healthcare Worker permissions.
3. Session expiry after 30 minutes; logout after 5 failed attempts.
4. AES-256 encryption at rest; TLS 1.3 in transit.
5. Full audit logging with timestamps, user IDs, and IP addresses.

1.4 Validity and Effectiveness of Proposed Solution

1.4.1 Competitive Analysis

Table 1.1: Competitive Analysis: SkinSense vs. Existing Solutions

Feature	SkinVision	FirstDerm	Skinive	DermEngine	SkinSense
Offline Capable	No	No	No	No	✓
Edge Inference	No	No	No	No	✓
Explainable AI	No	No	Limited	No	✓
Multilingual (Urdu)	No	No	No	No	✓
Voice Input	No	No	No	No	✓
Human-in-Loop	No	Partial	No	No	✓
Cost per Screening	\$5–10	\$30–50	\$15–25	Enterprise	<\$0.01

1.4.2 Justification for PWA over Native Mobile App

SkinSense is deployed as a Progressive Web Application rather than a native Android or iOS application for the following evidence-based reasons:

1. **Storage constraints:** Target users operate on entry-level Android devices with 16–32GB storage often near capacity. A native app requires 50–100MB installation plus model storage.
2. **Zero-installation access:** A PWA loads in any browser and after the first visit operates fully offline via service worker caching, eliminating the need for app store access.
3. **Cross-device compatibility:** Works on any device with a browser, not locked to Android or iOS.
4. **Research precedent:** A peer-reviewed study in *Frontiers in Public Health* (2021) demonstrated ML clinical triage tools deployed as offline PWAs function seamlessly under intermittent connectivity in resource-limited settings [4].

1.5 User Classes and Characteristics

Table 1.2: User Classes and Characteristics

User Class	Description
Patient / End User	Individuals experiencing skin symptoms. May be urban or rural with varying digital literacy, low-end smartphones, intermittent connectivity, and Urdu language preference. The system provides a simplified interface, voice-assisted interaction, and probabilistic outputs rather than definitive diagnoses.
Healthcare Worker	Community health workers, nurses, or field practitioners. Require quick results, low-latency performance, and clear visual explanations to guide referral decisions. Have moderate digital literacy and basic medical training but are not dermatology specialists.

Chapter 2

Project Requirements

2.1 Use Cases and Use Case Diagram

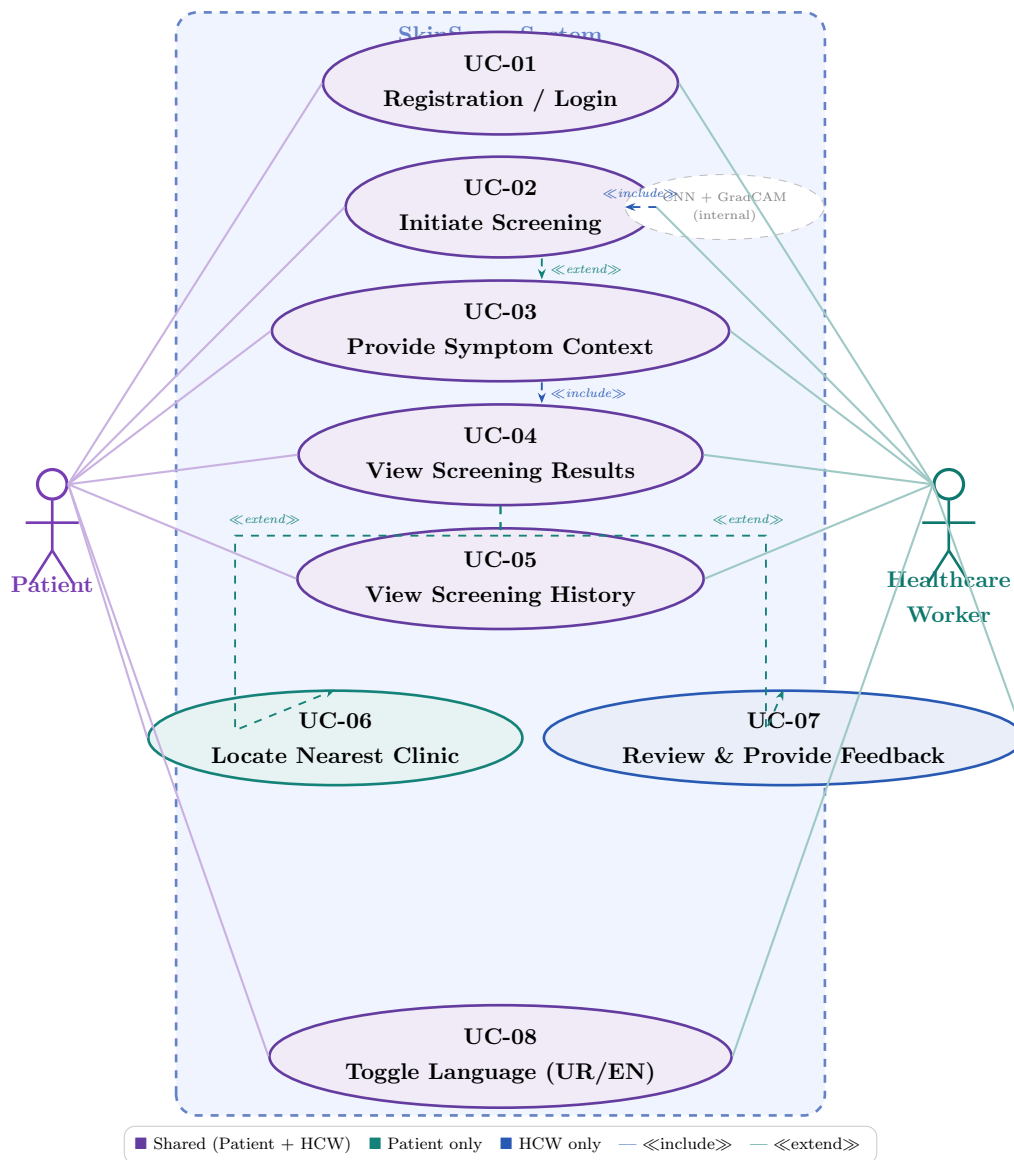


Figure 2.1: SkinSense Use Case Diagram

2.1.1 Detailed Use Cases

UC-01: Registration / Login

Primary Actor: Patient / Healthcare Worker

Description: The user registers a new account or logs into an existing one.

Preconditions: System is operational. User has access to the web interface.

Main Flow:

1. User navigates to the login or registration page.
2. User selects their role (Patient or Healthcare Worker).
3. User enters credentials.
4. System validates credentials and checks role-based permissions.
5. System issues JWT token and grants access to the role-appropriate portal.

Postconditions: Authenticated session established.

Alternative Flow: After 5 consecutive failed attempts, account locked for 15 minutes.

UC-02: Initiate Dermatology Screening

Primary Actor: Patient / Healthcare Worker

Description: The user uploads or captures a skin image to begin screening. CNN inference and GradCAM are triggered automatically.

Preconditions: User is authenticated.

Main Flow:

1. User selects “Start New Screening”.
2. User captures or uploads a skin image (JPEG/PNG).
3. System validates image quality (resolution, blur, lighting, format).
4. System creates a new ScreeningCase record.
5. System preprocesses the image and runs CNN inference (*includes CNN + Grad-CAM*).
6. System stores results and notifies the clinician dashboard.

Postconditions: Screening case created. Results available. Clinician notified.

Alternative Flow: Image fails quality validation — user prompted to retake.

UC-03: Provide Symptom Context (Voice / Text)

Primary Actor: Patient / Healthcare Worker

Description: The user optionally provides symptom information via voice or text.

Preconditions: Screening session active (extends UC-02).

Main Flow:

1. User selects voice or text input mode.
2. If voice: user records in Urdu or English (maximum 30 seconds).
3. System sends audio to Whisper ASR for transcription.
4. System extracts symptom keywords via NLP.
5. System attaches context to the active ScreeningCase.

Postconditions: Symptom context stored with screening case.

Alternative Flow: Confidence below 80% — data marked as low-confidence.

UC-04: View Screening Results

Primary Actor: Patient / Healthcare Worker

Description: The user views probabilistic confidence scores and GradCAM heatmap.

Preconditions: Inference completed (included from UC-02/03).

Main Flow:

1. System displays top predicted condition with confidence percentage.
2. System displays full probability distribution across all seven conditions.
3. System overlays GradCAM heatmap on original image.
4. System displays colour-coded confidence indicator (Red <60%, Yellow 60–80%, Green >80%).
5. System displays disclaimer: “This is a preliminary screening. Please consult a dermatologist.”

Postconditions: User has reviewed preliminary screening output.

UC-05: View Screening History

Primary Actor: Patient / Healthcare Worker

Description: The user views all past screening cases with results.

Preconditions: User is authenticated.

Main Flow:

1. User navigates to screening history section.
2. System retrieves all ScreeningCase records for the user.
3. System displays list with date, top condition, confidence, and status.
4. User selects a case to view full details.

Postconditions: Historical data visible to the user.

UC-06: Locate Nearest Clinic

Primary Actor: Patient / Healthcare Worker

Description: The user finds the nearest healthcare facility. Available independently or as an extension of UC-04.

Preconditions: User is authenticated.

Main Flow:

1. User navigates to clinic finder or selects from results page.
2. System requests location access or user selects area manually.
3. System queries Map API with location and specialisation filters.
4. System displays nearest clinics with distance and contact information.

Postconditions: User receives referral guidance.

UC-07: Review and Provide Feedback on Screening Case

Primary Actor: Healthcare Worker

Description: The clinician reviews screening output and submits a human-in-the-loop decision.

Preconditions: Healthcare Worker is authenticated. At least one pending case exists.

Main Flow:

1. Clinician views pending cases on doctor portal dashboard.
2. Clinician selects a case.
3. System displays full case: image, GradCAM, confidence scores, symptoms.
4. Clinician selects: **Accept**, **Dispute**, or **Correct**.
5. Clinician optionally adds notes.
6. System stores feedback, updates status to “reviewed”, logs to audit trail.
7. If disputed or corrected, case is flagged for retraining.

Postconditions: Feedback recorded. Case status updated. Audit entry created.

Alternative Flow: “Correct” requires corrected_diagnosis before submission.

UC-08: Toggle Language (Urdu / English)

Primary Actor: Patient / Healthcare Worker

Description: The user switches UI language between Urdu and English.

Preconditions: User has access to the web interface.

Main Flow:

1. User selects language toggle from navigation.
2. System re-renders all UI text within 500ms.
3. Language preference persisted for the session.

Postconditions: UI displayed in chosen language. Voice input defaults to match.

2.2 Functional Requirements

2.2.1 Module 1: Patient Portal

1. The system shall allow patients to register and log in using JWT-based authentication.
2. The system shall allow patients to upload or capture skin images for screening.
3. The system shall validate image quality before inference and prompt retake on failure.
4. The system shall display probabilistic confidence scores across all seven conditions.
5. The system shall overlay GradCAM heatmaps on screening images.
6. The system shall provide multilingual support for Urdu and English.
7. The system shall support voice-based symptom input in Urdu and English.
8. The system shall display the nearest clinic based on user location via Map API.
9. The system shall function fully offline after initial model download.

2.2.2 Module 2: Healthcare Worker Portal

1. The system shall allow Healthcare Workers to log in with role-based access.
2. The system shall display detailed screening output with full confidence distribution.

3. The system shall display GradCAM heatmaps for clinical review.
4. The system shall allow Healthcare Workers to accept, dispute, or correct results.
5. The system shall maintain a full case review history.
6. The system shall log all Healthcare Worker actions for audit purposes.

2.2.3 Module 3: AI Inference Module

1. The system shall perform inference using a distilled MobileNetV3-Large model.
2. The system shall generate probabilistic confidence scores for all seven conditions.
3. The system shall support offline operation after initial model download.
4. The system shall generate GradCAM heatmaps for all inference results.
5. The system shall switch to local inference within 3 seconds if cloud is unavailable.

2.2.4 Module 4: Speech and NLP Module

1. The system shall convert Urdu and English speech to text using Whisper v3 Turbo Large.
2. The system shall extract symptom keywords from transcribed text.
3. The system shall attach extracted symptoms to the active screening case.
4. The system shall implement audio fallback to ensure zero data loss if WAV conversion fails.

2.3 Non-Functional Requirements

2.3.1 Performance Requirements

Table 2.1: Performance Requirements for SkinSense

ID	Requirement
PERF-01	Inference Latency: The system shall complete screening within 5 seconds per image on devices with minimum 2GB RAM and 1.4GHz quad-core, online and offline.
PERF-02	Concurrent Users: The system shall support 50 concurrent users on the cloud backend without response time exceeding 8 seconds per request.
PERF-03	Low-Bandwidth: The system shall remain fully functional on connections as slow as 2 Mbps. Model download shall complete within 10 minutes on a 2 Mbps connection.
PERF-04	ASR Latency: Voice transcription shall complete within 10 seconds for recordings up to 30 seconds.

2.3.2 Reliability Requirements

Table 2.2: Reliability Requirements for SkinSense

ID	Requirement
REL-01	Availability: Cloud backend shall maintain 99% uptime monthly. Offline mode fully operational during outages up to 12 hours.
REL-02	Data Integrity: All records stored with SHA-256 checksums. Corruption triggers automatic re-upload and admin alert.
REL-03	Backup: Local data syncs to cloud within 60 seconds of connectivity restoration. Maximum one session of data lost on device failure.
REL-04	Model Consistency: AI model shall produce bit-identical predictions for identical inputs (deterministic, fixed random seed).
REL-05	Offline Fallback: System shall switch to local inference within 3 seconds if cloud unavailable, with no user intervention.

2.3.3 Usability Requirements

Table 2.3: Usability Requirements for SkinSense

ID	Requirement
USE-01	Multilingual: UI shall fully render in Urdu or English within 500ms of language toggle with no untranslated strings.
USE-02	Voice Accuracy: ASR shall achieve minimum 85% word-level accuracy on Urdu and English medical symptoms on a 100-sample test set.
USE-03	Error Clarity: All errors shall be in the user’s selected language, include a plain-language description, and suggest a corrective action.
USE-04	Offline Indicator: System shall display offline status within 2 seconds of losing connectivity and restoration notification within 2 seconds of reconnecting.
USE-05	Learnability: A first-time user with no digital health experience shall complete a full screening within 3 minutes, validated through UAT.

2.3.4 Security Requirements

Table 2.4: Security Requirements for SkinSense

ID	Requirement
SEC-01	Authentication: Passwords minimum 8 characters with uppercase, number, and special character. Hashed with bcrypt cost factor ≥ 12 .
SEC-02	Authorisation: RBAC enforces strict separation between Patient and Healthcare Worker permissions.
SEC-03	Encryption: Patient data encrypted at rest with AES-256. All transit uses TLS 1.3 exclusively.
SEC-04	Audit Logging: All user actions logged with timestamp, user ID, and IP address. Logs retained minimum 90 days.
SEC-05	Session Management: Sessions expire after 30 minutes inactivity. Account locked 15 minutes after 5 failed attempts.

Chapter 3

System Overview

SkinSense allows a user to capture or upload an image of a skin condition and optionally provide symptom information through voice or text. Using a knowledge-distilled deep learning model, the system produces probabilistic screening results for seven dermatological conditions. The system is a clinical decision support tool, not a diagnostic authority. GradCAM heatmaps highlight the image regions that influenced the model’s prediction. The platform includes two role-based interfaces: a patient interface and a clinician dashboard. SkinSense is optimised for resource-constrained environments with lightweight models, low computational requirements, and offline-first operation.

3.1 Architectural Design

3.1.1 Architecture Selection and Justification

SkinSense employs a **layered (N-tier) architecture**. This section provides a formal justification over alternative patterns.

Microservices was considered but rejected. While it enables independent scaling and fault isolation, it requires service meshes, container orchestration, and distributed tracing — engineering overhead disproportionate to a three-person FYP team, and unnecessary given that SkinSense does not require independent scaling of individual components.

Event-Driven Architecture was considered but rejected. SkinSense’s core workflow is inherently synchronous — a patient uploads an image and expects a result within 5 seconds. Asynchronous message queues add latency without addressing any real requirement.

Client-Server (2-Tier) provides insufficient separation of concerns for a system requiring distinct security boundaries between auth, routing, ML inference, and data persistence.

MVC governs the frontend (Next.js) but does not naturally express the multi-service backend, dual online/offline inference modes, or edge-first deployment model central to SkinSense’s design.

Layered architecture was selected for three specific reasons:

1. **Security boundaries:** Auth and access control sit at the API Gateway layer, ensuring business logic and data are never directly accessible from the presentation layer — critical for patient data privacy.
2. **Clean service separation:** Express handles auth and routing while FastAPI handles ML inference, cleanly separated within the Business Logic layer. The frontend never communicates with FastAPI directly.
3. **Offline-first design expression:** The Business Logic layer explicitly contains the Model Manager governing local model caching, mode detection, and background download — making the offline-first architecture visible at the design level.

3.1.2 Architectural Diagram

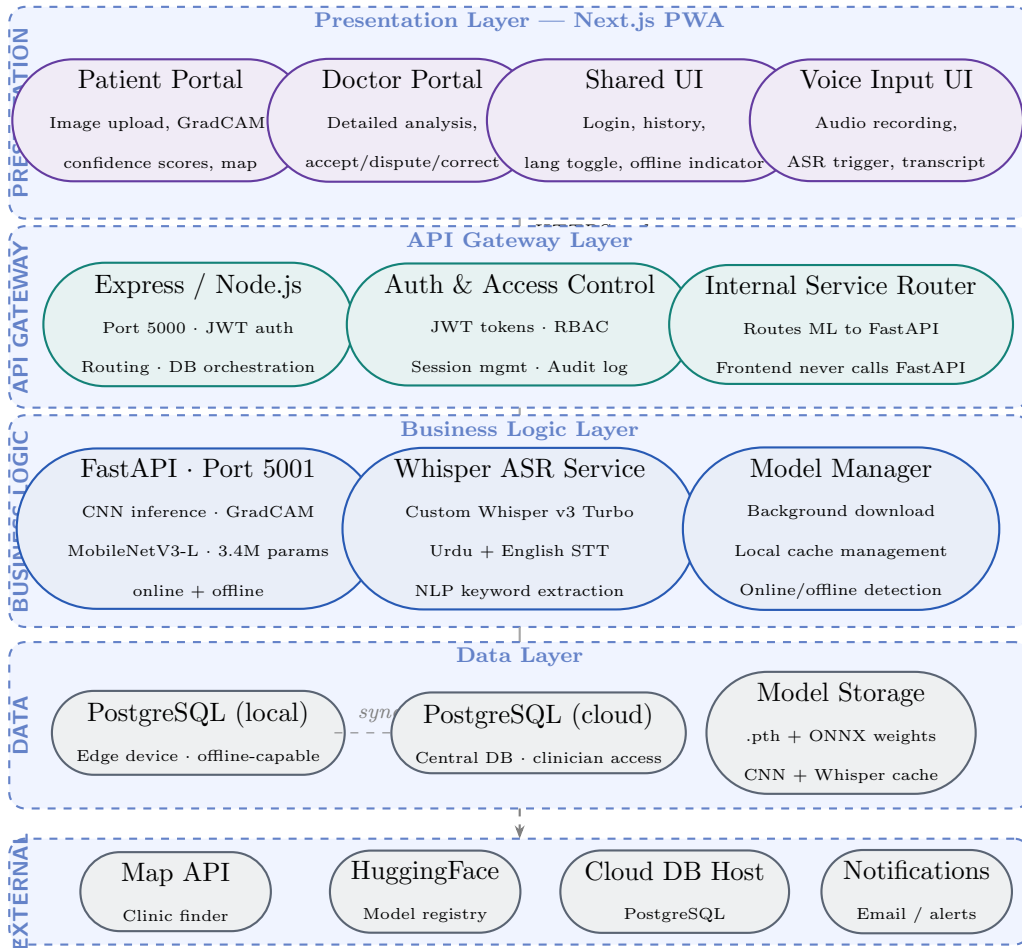


Figure 3.1: SkinSense Five-Layer Logical Architecture

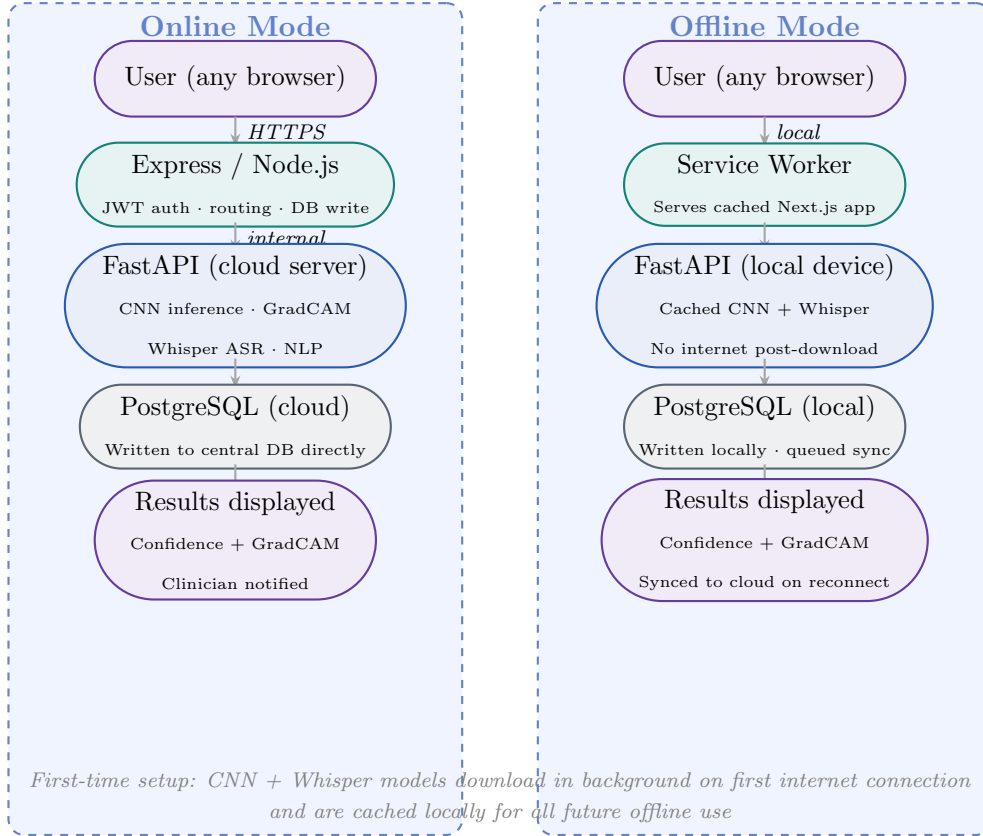


Figure 3.2: SkinSense Deployment Modes: Online vs Offline

3.1.3 Layer Descriptions

Presentation Layer

Implemented as a Next.js PWA: Patient Portal (image upload, GradCAM overlay, confidence scores, nearest clinic map), Doctor Portal (detailed analysis, accept/dispute/correct feedback), Shared UI (login, history, language toggle, offline indicator), and Voice Input UI (audio recording, ASR trigger, transcript display).

API Gateway Layer

Express/Node.js (Port 5000) handles JWT authentication, routing, and database orchestration. Auth & Access Control enforces RBAC and manages sessions. The Internal Service Router proxies all ML requests to FastAPI — the frontend never communicates with FastAPI directly.

Business Logic Layer

FastAPI (Port 5001) handles image preprocessing, CNN inference (MobileNetV3-Large distilled, 3.4M parameters, 71% accuracy), and GradCAM generation. The Whisper ASR Service (custom Whisper v3 Turbo Large from HuggingFace) handles Urdu and

English speech-to-text and NLP extraction. The Model Manager governs background model download, local caching, and online/offline mode detection.

Data Layer

Local PostgreSQL stores screening data on the edge device for offline operation. Cloud PostgreSQL is the central database for clinician access and cross-device synchronisation (within 60 seconds of connectivity restoration). Model Storage holds locally cached CNN and Whisper weights.

External Services Layer

Map API (nearest clinic finder), HuggingFace (model weight registry), Cloud PostgreSQL host, and email/notification service for clinician alerts.

3.2 Design Models

3.2.1 Sequence Diagrams

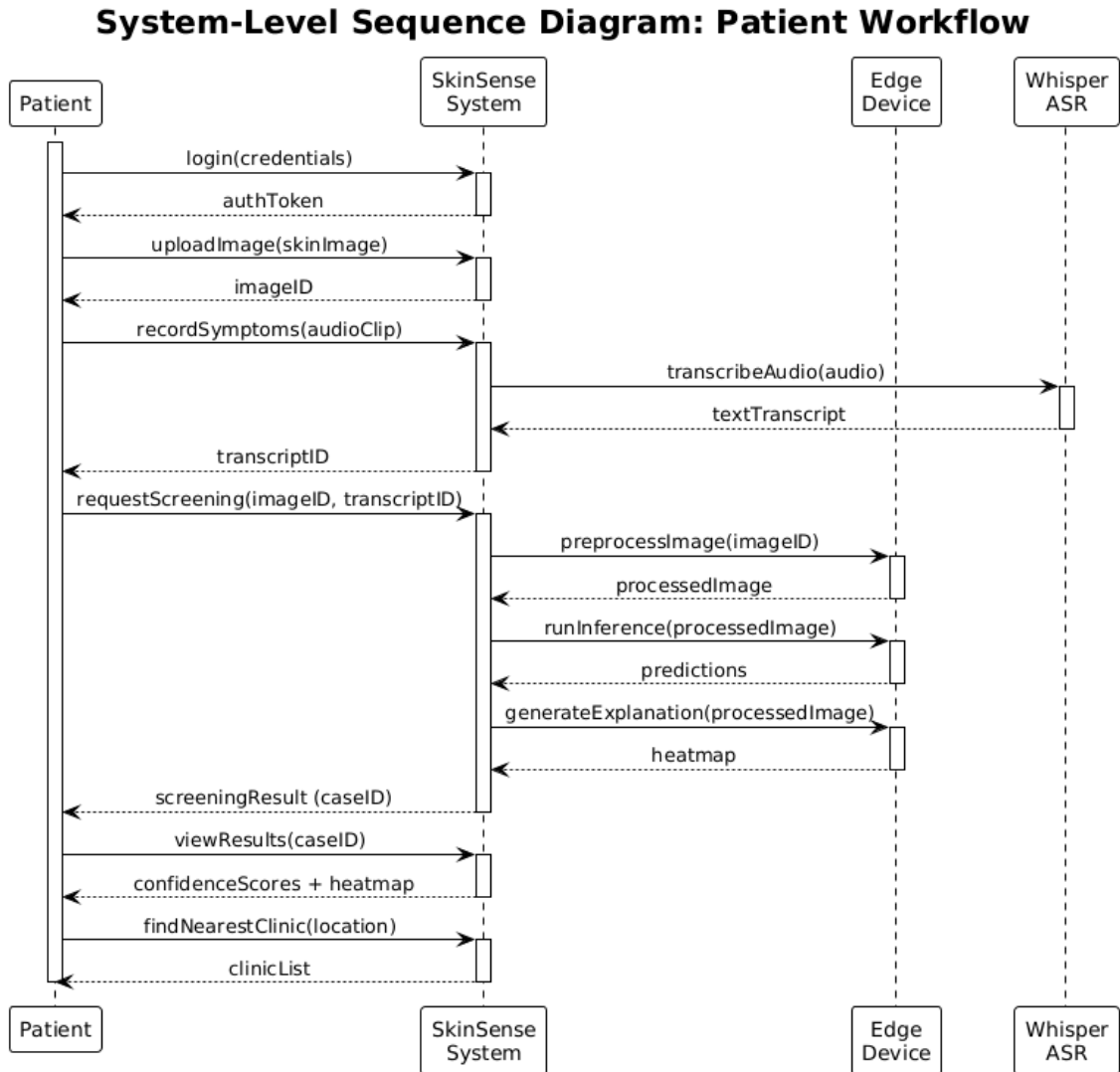


Figure 3.3: System-Level Sequence Diagram: Patient Screening Workflow

System-Level Sequence Diagram: Clinician Workflow

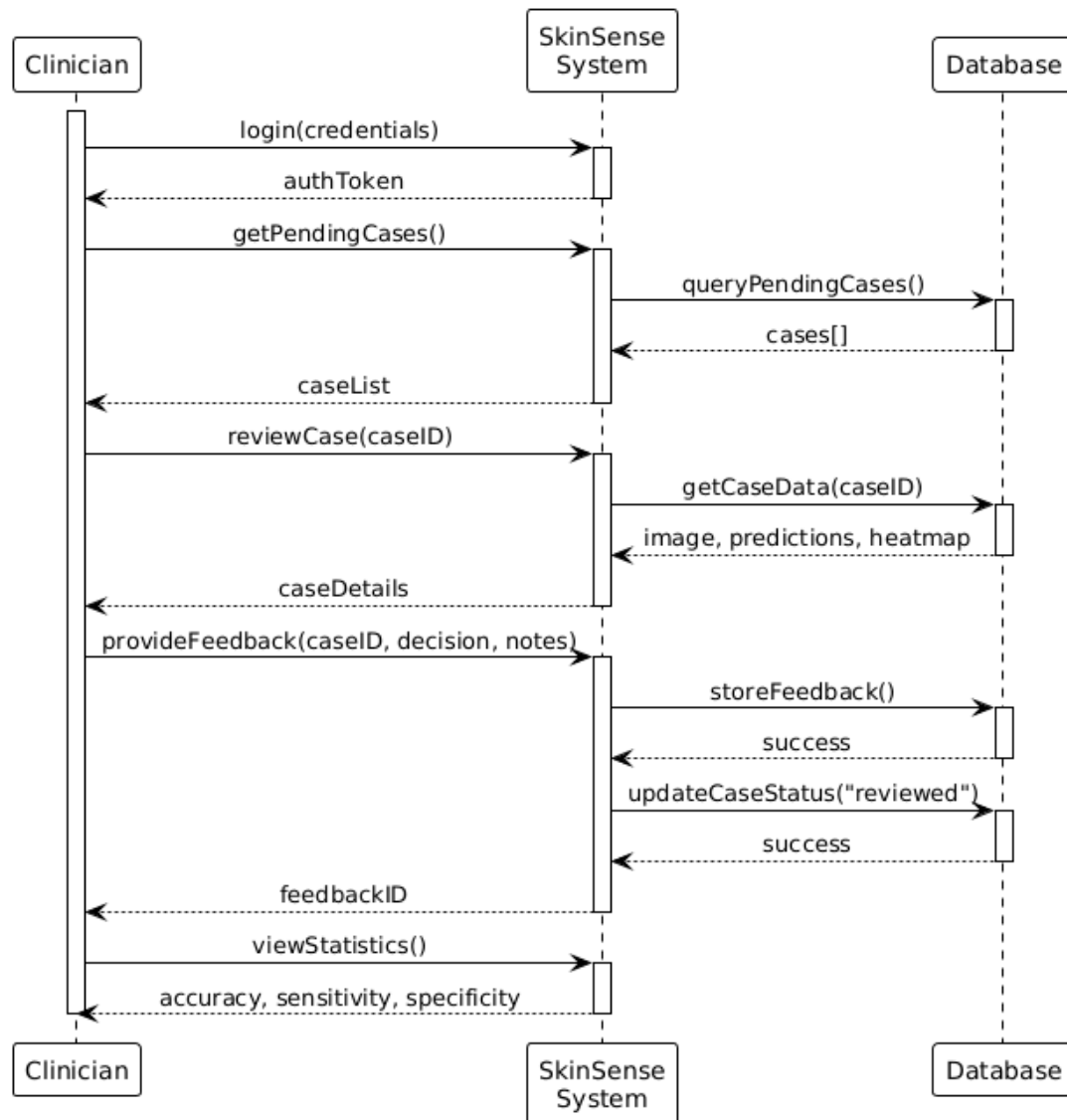


Figure 3.4: System-Level Sequence Diagram: Clinician Feedback Workflow

3.2.2 Activity Diagram

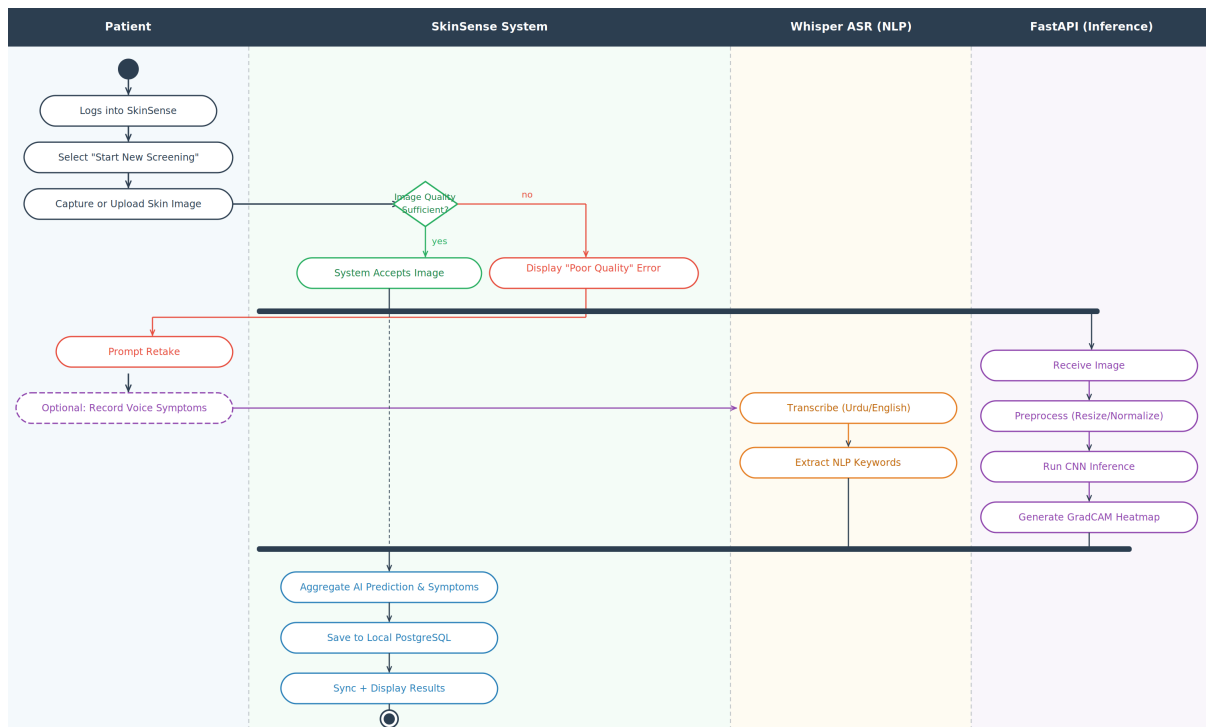


Figure 3.5: Activity Diagram: Complete Patient Screening Workflow with Swimlanes

3.2.3 Class Diagram

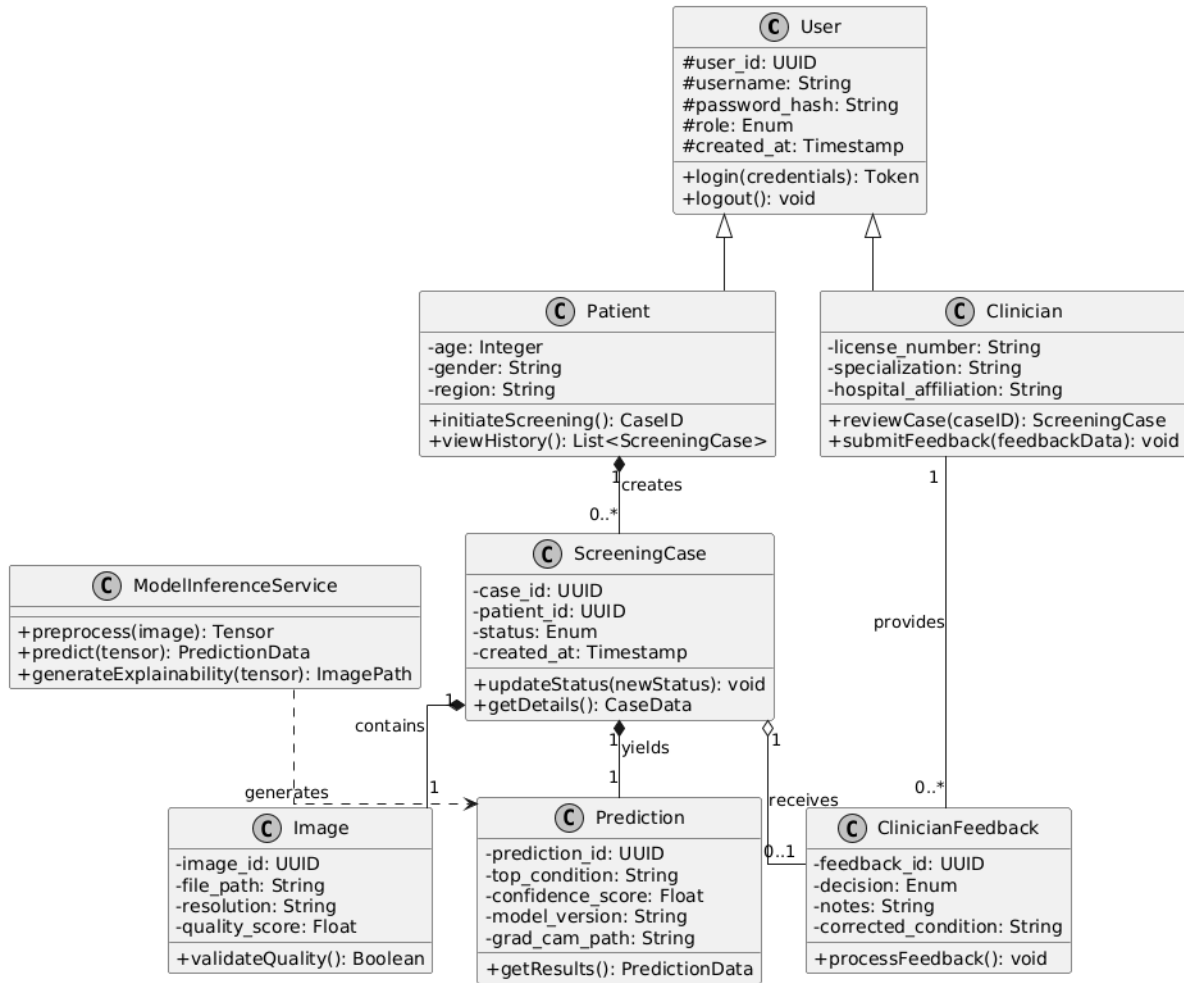


Figure 3.6: Class Diagram for SkinSense System

3.2.4 Error Handling and Alternative Flows

Table 3.1: Error Handling in Screening Workflow

Error Condition	Detection Point	Recovery Action
Poor image quality	Quality validation	Prompt retake with guidance
Invalid image format	Format validation	Display error, request correct format
ASR confidence <80%	ASR output	Continue; mark as low-confidence
AI inference failure	CNN execution	Return structured 503; log error
WAV conversion failure	AudioContext	Fall back to raw blob transmission
Cloud sync failure	Network check	Queue for later sync; use local DB
Malformed JWT	API Gateway	Return strict 401 response

3.3.2 Entity Relationships and Cardinalities

Table 3.2: Entity Relationships and Cardinalities

Relationship	Cardinality	Description
User — PatientProfile	1:0..1	User may have one patient profile
User — ClinicianProfile	1:0..1	User may have one clinician profile
PatientProfile — ScreeningCase	1:*	Patient can have multiple cases
ScreeningCase — Image	1:1	Each case has exactly one image
ScreeningCase — VoiceTranscript	1:0..1	Case may have optional transcript
ScreeningCase — Prediction	1:1	Each case has one prediction
VoiceTranscript — ExtractedSymptom	1:*	Transcript has multiple symptoms
ScreeningCase — ClinicianFeedback	1:0..*	Case may receive multiple feedbacks
ClinicianProfile — ClinicianFeedback	1:*	Clinician provides multiple feedbacks
User — AuditLog	1:*	User has multiple audit log entries

3.3.3 Database Indexes

Table 3.3: Database Indexes for Performance

Table	Indexed Columns	Purpose
User	username, email	Fast user lookup
ScreeningCase	patient_id, created_at	Patient history retrieval
ScreeningCase	status	Filter pending/reviewed
Prediction	model_version, created_at	Model performance tracking
ClinicianFeedback	case_id, clinician_id	Fast feedback retrieval
AuditLog	user_id, timestamp	User activity tracking

Chapter 4

ML Pipeline

4.1 Overview

The SkinSense ML pipeline was developed through an iterative process of dataset integration, class imbalance correction, and progressive model improvement via knowledge distillation. The pipeline targets seven conditions: Vitiligo, Melasma, Psoriasis, Eczema, Tinea, Contact Dermatitis, and Seborrheic Dermatitis. All model decisions were driven by the deployment constraint of running on devices with a minimum of 2GB RAM with total model footprint under 50MB after quantisation.

4.2 Dataset Curation

4.2.1 Selection Criteria

Datasets were selected based on: relevance to the seven target classes, skin tone diversity (prioritising South Asian Fitzpatrick Scale 4–6 representation), image quality, label reliability, and public availability.

4.2.2 Datasets Used

Table 4.1: Dataset Sources and Contributions

Dataset	Images	Source	Key Notes
DermaCon-IN	2,688	IIT Hyderabad	Primary South Asian dataset. Most representative of target population.
SCIN	889	Google Research	Designed for diverse skin tones including FST 4–6. Fitzpatrick metadata available.
SkinDisNet	1,297	Mendeley Data	Bangladesh clinical images. Moderate South Asian representation.
DermNet	625	Kaggle	Western-biased. Psoriasis folder contained mixed conditions requiring file-name regex filtering.
SkinDiseaseImage	6,691	Kaggle	Largest contributor. Western-biased. Folder 8 (Seborrheic Keratoses) excluded.
Total	12,190		

4.2.3 Final Dataset Composition

Table 4.2: Final Dataset Composition by Class

Class	DermaCon	SCIN	SkinDisNet	DermNet	SkinDisImg	Total
Vitiligo	601	3	0	16	0	620
Melasma	80	5	0	8	0	93
Psoriasis	227	126	0	141	2,055	2,549
Eczema	256	470	466	432	2,934	4,558
Tinea	1,289	154	275	0	1,702	3,420
Contact Dermatitis	175	124	477	27	0	803
Seborrheic Dermatitis	60	7	79	25	0	171
Total	2,688	889	1,297	625	6,691	12,190

4.2.4 Train / Validation / Test Split

A stratified split was applied to preserve class distribution across all partitions:

- **Train:** 7,801 images (64%)
- **Validation:** 1,951 images (16%)
- **Test:** 2,438 images (20%)

Class imbalance was addressed using `WeightedRandomSampler` with inverse frequency class weights. A critical finding was that applying both weighted sampling and class-weighted loss simultaneously caused training failure (validation accuracy collapsed to $\sim 40\%$). Weighted sampling must replace, not supplement, class-weighted loss.

4.3 Model Training Iterations

4.3.1 Baseline: MobileNetV3-Small

MobileNetV3-Small (1M parameters) was selected as the baseline for its mobile/edge deployment design. Two-phase training: frozen backbone (lr=0.001, 20 epochs) followed by full fine-tuning (lr=0.0001, 20 epochs).

Table 4.3: Baseline MobileNetV3-Small Per-Class Test Results

Class	Precision	Recall	F1-Score
Vitiligo	0.71	0.78	0.74
Melasma	0.41	0.94	0.57
Psoriasis	0.44	0.79	0.56
Eczema	0.75	0.54	0.63
Tinea	0.75	0.50	0.60
Contact Dermatitis	0.46	0.49	0.47
Seborrheic Dermatitis	0.25	0.44	0.32
Overall Accuracy	0.59		

4.3.2 Teacher Model: EfficientNet-B3

EfficientNet-B3 (11.4M parameters, 300×300 input) was selected as the teacher. A critical finding was that the frozen-backbone approach that works for MobileNetV3 causes catastrophic BatchNorm instability in EfficientNet. Five training attempts were required to identify the correct configuration: full unfreeze from epoch 1, lr= 5×10^{-5} , weight_decay= 10^{-4} , label_smoothing=0.1, early stopping patience=3.

Table 4.4: EfficientNet-B3 Teacher Model Per-Class Test Results

Class	Precision	Recall	F1-Score
Vitiligo	0.84	0.88	0.86
Melasma	0.54	0.88	0.67
Psoriasis	0.68	0.64	0.66
Eczema	0.73	0.78	0.76
Tinea	0.79	0.67	0.73
Contact Dermatitis	0.50	0.65	0.56
Seborrheic Dermatitis	0.39	0.35	0.37
Overall Accuracy		0.71	

4.3.3 Knowledge Distillation

Knowledge distillation transfers the teacher’s soft probability distributions to the student model, providing a richer training signal than hard one-hot labels. The distillation loss is:

$$\mathcal{L} = \alpha \cdot \text{KL}\left(\sigma\left(\frac{z_s}{T}\right), \sigma\left(\frac{z_t}{T}\right)\right) \cdot T^2 + (1 - \alpha) \cdot \text{CE}(z_s, y) \quad (4.1)$$

where z_s and z_t are student and teacher logits, $T = 4$ is the temperature, $\alpha = 0.7$ weights the distillation loss, and y are the hard labels.

A `DualTransformDataset` was implemented to synchronise dual-size inputs: each image is opened once with both student (224×224) and teacher (300×300) transforms applied, returning a tuple per sample.

4.3.4 Final Model: MobileNetV3-Large (Distilled)

Table 4.5: MobileNetV3-Large Distilled Student Per-Class Test Results

Class	Precision	Recall	F1-Score
Vitiligo	0.78	0.88	0.83
Melasma	0.57	0.94	0.71
Psoriasis	0.66	0.68	0.67
Eczema	0.74	0.74	0.74
Tinea	0.77	0.68	0.72
Contact Dermatitis	0.51	0.61	0.55
Seborrheic Dermatitis	0.39	0.38	0.39
Overall Accuracy	0.71		

4.4 Model Comparison Summary

Table 4.6: Full Model Comparison

Model	Parameters	Test Acc.	Size
MobileNetV3-Small (baseline)	1.0M	59%	~10MB
MobileNetV3-Small (distilled)	1.0M	67%	~10MB
EfficientNet-B3 (teacher)	11.4M	71%	~48MB
MobileNetV3-Large (distilled)	3.4M	71%	~14MB

The distilled MobileNetV3-Large matches teacher accuracy (71%) at 30% of the teacher’s parameter count. Knowledge distillation yielded an 8% accuracy improvement over the baseline with zero inference cost increase.

4.5 Key Findings

1. **Data quality over quantity:** DermaCon-IN’s 2,688 South Asian clinical images contributed more to real-world performance than SkinDiseaseImage’s 6,691 Western-biased images.
2. **Weighted sampling must replace weighted loss:** Applying both simultaneously caused complete training failure.

3. **EfficientNet requires full unfreeze:** Frozen-backbone approach causes catastrophic BatchNorm instability in EfficientNet regardless of learning rate.
4. **Knowledge distillation delivers +8% at zero inference cost.**
5. **Seborrheic Dermatitis is a data problem:** F1 barely moved from 0.37 to 0.39 across all models. Only clinical data collection will resolve this.

Chapter 5

Testing and Quality Assurance

5.1 Testing Overview

SkinSense underwent comprehensive automated testing across all three services that constitute the system: the Express/Node.js backend API, the React frontend, and the Python FastAPI inference service. A total of **193 tests** were executed across 10 test files; all 193 pass in the current codebase, with no tests skipped or conditionally disabled. Testing was designed to satisfy three requirements for every functional module:

1. **Success paths.** The system produces the correct output for valid, well-formed input.
2. **Error paths.** The system returns the correct HTTP status code and a descriptive error message for invalid or rejected input.
3. **Edge cases.** The system behaves correctly at boundary conditions and in failure scenarios such as network unavailability, corrupted data, and resource exhaustion.

5.2 Testing Infrastructure

5.2.1 Technology Stack

Each service uses a test framework idiomatic to its runtime, with HTTP-level testing performed through transport-aware clients that exercise the real request handlers without binding to a network port.

Table 5.1: Testing Frameworks by Service

Service	Test Framework	HTTP Testing	Assertion Library	Li-
Express backend	Jest 29	Supertest	Jest matchers	built-in
React frontend	Vitest 3	N/A (hook tests)	Vitest + Testing Library	
Python inference	pytest 9	FastAPI TestClient	pytest assert	

5.2.2 Test Environment Setup

Server. Before any test module loads, Jest executes `server/tests/setup.js`, which configures three environment variables: `JWT_SECRET` is set to a fixed test value so token signing is deterministic without requiring a `.env` file; `UPLOAD_DIR` is pointed at a real OS temporary directory so multer can write files without permission errors; and `JWT_EXPIRES_IN` is set to one hour to prevent token expiry during test execution.

Client. Because the client tests run in jsdom (a headless browser simulation), several browser APIs are missing or incomplete. The client setup file `client/src/__tests__/setup.js` installs the following polyfills and mocks: a `Blob.prototype.arrayBuffer` polyfill so the WAV conversion pipeline inside `useVoiceRecorder` can exercise its normal code path; a `MockMediaRecorder` class that replaces the browser's `MediaRecorder` globally and fires synthetic `dataavailable` and `stop` events on `stop()`; `MockAudioContext` and `MockOfflineAudioContext` classes that implement `decodeAudioData` and `startRendering` returning mock buffers; `global.URL.createObjectURL` returning a fixed string so blob URLs do not throw; and `navigator.mediaDevices.getUserMedia` replaced with a controllable Vitest mock so tests can simulate either granted or denied microphone access.

Python inference. Environment variables must be set in `conftest.py` before `server.py` is imported, because model loading occurs at module level. `MODEL_PATH` is pointed at a nonexistent file so the CNN initialises with random weights: predictions are meaningless, but the response shape, status codes, and data types are fully exercisable without distributing the trained weights file. `ASR.MODEL_PATH` is similarly nonexistent, so the Whisper pipeline is `None` at startup and every `/transcribe` request returns 503, which is the documented contract for when the ASR model is unavailable. `HEATMAP_DIR` points to a real temporary directory so the `/predict` endpoint can write GradCAM PNG files to disk and the `/heatmaps` tests can read them back to verify their binary content. Three module-scoped fixtures are shared across all four Python test modules: a `TestClient` wrapping the FastAPI app, valid 10×10 PNG and JPEG byte buffers produced with Pillow, and a synthetically constructed 16 kHz mono WAV file containing 0.1 seconds of silence.

5.2.3 Mocking Strategy

Database. All server tests mock the PostgreSQL client with a Jest `jest.fn()`. Individual tests configure either `.mockResolvedValueOnce` (with the rows the controller should observe) or `.mockRejectedValueOnce` (with an `Error`) in sequence, simulating the exact database responses the controller under test expects. This approach tests controller logic in complete isolation from the database schema.

Rate limiters. All four rate limiters (`loginLimiter`, `registerLimiter`, `apiLimiter`, `strictLimiter`) are replaced with passthrough functions `(req, res, next) => next()`. This is necessary because the same endpoint is hit many times within a single test file, and the real rate limiter would return 429 after the first few requests.

bcrypt. The `bcryptjs` library is mocked to return a fixed hash string instantly. The real `bcrypt` function performs 12 rounds of key stretching, which adds approximately 300 ms of CPU time per hash. Mocking reduces the `auth.test.js` suite from over 10 seconds to under 1 second.

Axios (client). The Axios instance at `client/src/utils/api.js` is replaced with Vitest mock functions for `post` and `get`. Each test configures the mock to resolve or reject with specific response data, allowing hook state management to be tested without a running backend.

Python inference model. No explicit mocking is used for the model. Instead, `MODEL_PATH` is pointed at a nonexistent file so PyTorch loads the MobileNetV3-Large architecture with random weights. The API shape, numeric contracts (confidence in $[0, 1]$, scores summing to approximately 1.0), and file output (heatmap PNG written to disk) are fully verifiable with random weights.

5.3 Test Coverage by Service

The 193 tests are distributed across 10 files, with backend, frontend, and inference each receiving substantive coverage proportional to its surface area.

Table 5.2: Test Coverage Summary by File

Service	Test File	Tests	Result
Express backend	<code>auth.test.js</code>	28	✓
Express backend	<code>screening.test.js</code>	30	✓
Express backend	<code>middleware.test.js</code>	21	✓
React frontend	<code>useScreening.test.jsx</code>	21	✓
React frontend	<code>useVoiceRecorder.test.jsx</code>	15	✓
React frontend	<code>imageValidation.test.js</code>	23	✓
Python inference	<code>test_predict.py</code>	19	✓
Python inference	<code>test_transcribe.py</code>	9	✓
Python inference	<code>test_heatmap.py</code>	10	✓
Python inference	<code>test_utils.py</code>	17	✓
Total		193	193 / 193

The framework used by each service is summarised in Table 5.1.

5.4 Backend (Server) Tests

The Express backend test suite is located in `server/tests/` and contains three files covering authentication endpoints, the screening pipeline, and middleware behaviour. All controllers are tested against a mocked database; rate limiters are bypassed so multiple requests within a single suite do not trigger 429 responses.

5.4.1 Authentication Tests (`auth.test.js`)

This file contains 28 tests covering all four authentication endpoints: registration, login, logout, and profile retrieval. Coverage includes input validation, duplicate detection, credential verification, JWT lifecycle (issue, expire, tamper, revoke), and role-based response shaping.

Table 5.3: Authentication Endpoint Test Distribution

Endpoint	Success	Error	Edge	Total
POST /register	5	6	0	11
POST /login	1	3	1	5
POST /logout	1	2	1	4
GET /profile	3	5	0	8
Total	10	16	2	28

A representative selection of tests, drawn from across all four endpoints, is shown below.

Table 5.4: Selected Authentication Test Cases

ID	Category	Test Description and Key Assertion	HTTP
R-01	Success	Registers a patient with all profile fields. Body has <code>token</code> ; <code>user.role</code> is <code>patient</code> ; JWT decodes to expected <code>userId</code> .	201
R-05	Error	Rejects negative age (<code>age: -5</code>). No DB query is made; error message matches <code>/age/i</code> .	400
R-09	Success	Accepts boundary ages 1 and 120. Both return 201 without error.	201
R-10	Error	Returns 409 when username or email already exists. Error matches <code>/already taken/i</code> .	409
L-05	Edge	Constant-time error message: unknown user and wrong password produce identical body strings to prevent username enumeration.	401
LO-02	Edge	A blacklisted token is rejected on subsequent requests; error matches <code>/revoked/i</code> .	401
P-05	Error	Token created with <code>exp</code> in the past is rejected; error matches <code>/expired/i</code> .	401
P-06	Error	Tampered token (last 10 characters replaced) is rejected.	401
P-07	Error	Malformed Bearer header (<code>'Bearer '</code> with empty value) is rejected.	401

Two-layer age validation. The age field is validated at two independent layers. At the client layer, `RegisterPage.jsx` checks for non-integer, negative, zero, or over-120 values and displays an error without sending the request. At the server layer, `authController.js` performs the same check before any database query is made, so a malicious client bypassing the UI validation still receives a 400 response. Tests R-05

through R-09 exercise the server-side layer.

Username enumeration defence (test L-05). Returning different messages for “user not found” versus “wrong password” allows an attacker to enumerate valid usernames. Both conditions return the same ‘Invalid credentials’ string. The test issues two distinct requests (one with an unknown username, one with a correct username and wrong password) and asserts that the response body strings are byte-identical.

Token blacklist (tests LO-01, LO-02). The blacklist is implemented as an in-memory JavaScript Set in `server/src/models/tokenBlacklist.js`. The `beforeEach` hook calls `clearBlacklist()` before each test to prevent token contamination between cases. Test LO-02 issues a logout request, then reuses the same token on a protected route and asserts that the second request is rejected with a `/revoked/i` error.

5.4.2 Screening Pipeline Tests (`screening.test.js`)

This file contains 30 tests exercising the five-step screening workflow: case creation, image upload, voice processing, AI inference, and result retrieval, plus the patient history listing endpoint. A `MINIMAL_JPEG` buffer (a valid 1×1 pixel JPEG) and a `silentWav` buffer (a 44-byte WAV header with one silent sample) are defined at the module level for use across upload tests.

Table 5.5: Screening Pipeline Test Distribution

Endpoint	Success	Error	Total
POST <code>/screening/create</code>	1	2	3
POST <code>/screening/:id/upload-image</code>	2	4	6
POST <code>/screening/:id/voice</code>	2	4	6
POST <code>/screening/:id/inference</code>	1	4	5
GET <code>/screening/:id/results</code>	3	3	6
GET <code>/screening/history/list</code>	2	2	4
Total	11	19	30

Graceful ASR fallback (test V-01). The test intentionally does not mock the Python ASR service. When the Express controller issues a `fetch` call to the local transcribe endpoint, the connection is refused (`ECONNREFUSED`) because no service is listening on port 5001 in the test environment. The controller’s catch block intercepts this, stores

a `null` transcript, and sets `asr_available: false` in the response. This verifies the resilience path: the screening flow continues even when ASR is offline.

Codec suffix tolerance (test V-02). Chrome’s `MediaRecorder` produces files with MIME type `audio/webm; codecs=opus`, which includes a codec parameter after the semi-colon. The multer audio filter uses `mimeType.startsWith('audio/')` rather than an exact match so this format is accepted. A naive equality check would silently reject every recording made in Chrome.

Inference fallback on missing file (test IN-01). Like ASR, the inference fallback is exercised through natural failure rather than a mock. When the recorded `file_path` does not exist on disk, the controller’s call to the Python service fails and a deterministic mock prediction (all seven conditions assigned equal probability) is stored instead. This verifies system resilience without requiring the inference service to be running during backend tests.

Multer MIME rejection (tests UI-04, V-04). Multer is configured with a MIME filter that calls `cb(new Error('Only JPEG and PNG images are allowed'), false)` for any MIME type other than `image/jpeg` or `image/png`. When multer calls the callback with an error, Express routes the request to the global error handler, which returns 500. Tests UI-04 (PDF submitted to image endpoint) and V-04 (PNG submitted to audio endpoint) verify this contract from both sides.

5.4.3 Middleware Tests (`middleware.test.js`)

This file contains 21 tests covering the authentication middleware, role-based authorisation middleware, and the multer upload MIME filters. Each middleware is mounted on a minimal Express application that exposes a single probe route; the tests then hit the probe and assert on the response, isolating middleware behaviour from controller logic.

Table 5.6: Middleware Test Distribution

Middleware	Success	Error	Total
<code>authenticate</code>	1	7	8
<code>requireRole</code>	1	3	4
<code>imageUpload</code> (MIME filter)	2	2	4
<code>audioUpload</code> (MIME filter)	3	2	5
Total	7	14	21

The `authenticate` middleware tests cover seven distinct rejection paths in addition to the success path: missing header, wrong scheme (Basic instead of Bearer), wrong signing secret, structurally invalid token, expired token, blacklisted token, and a token presented without the 'Bearer ' prefix. The `requireRole` suite verifies that role mismatches return 403 and that an unauthenticated request to a role-protected route returns 401 (not 403), because `authenticate` is mounted before `requireRole` and its rejection takes precedence.

5.5 Frontend (Client) Tests

The React frontend test suite is located in `client/src/__tests__` and covers the two primary custom hooks (`useScreening` and `useVoiceRecorder`) and the image validation utility module. Hooks are instantiated via `renderHook(() => useHook())` from React Testing Library; pure utility functions are tested directly without rendering.

5.5.1 Screening Hook Tests (`useScreening.test.jsx`)

`useScreening` manages the entire five-step screening workflow as a single React hook, exposing `createCase`, `uploadImage`, `submitVoice`, `runInference`, `getResults`, `getHistory`, and `reset`. The 21 tests distribute as follows.

Table 5.7: Screening Hook Test Distribution

Function	Success	Error	Edge	Total
<code>createCase()</code>	2	2	2	6
<code>uploadImage()</code>	1	1	0	2
<code>submitVoice()</code>	1	1	4	6
<code>runInference()</code>	1	1	0	2
<code>getResults()</code>	1	1	0	2
<code>getHistory()</code>	1	1	0	2
<code>reset()</code>	1	0	0	1
Total	8	7	6	21

File extension selection (tests SV-02 to SV-04). The hook chooses a filename extension based on the audio blob's MIME type: `audio/wav` produces `recording.wav`, `audio/webm` produces `recording.webm`, and `audio/ogg` produces `recording.ogg`. The tests confirm that the codec suffix in `audio/webm;codecs=opus` is correctly stripped before extension determination.

Loading state lifecycle (test UC-05). The test passes a deferred Promise to the Axios mock, asserts that `loading` is `true` while the request is pending, resolves the Promise, and asserts that `loading` flips to `false`. This verifies that the hook correctly drives the spinner UI throughout the request lifecycle.

Error state cleared on retry (test UC-06). The first call fails and sets `error`; the second call succeeds. The test asserts that `error` is `null` after the second call, confirming that residual error state from a previous attempt does not leak into the user's view of a successful retry.

5.5.2 Voice Recorder Hook Tests (`useVoiceRecorder.test.jsx`)

`useVoiceRecorder` manages microphone access, recording state, duration counting, WAV conversion, and cleanup. The 15 tests are organised into five describe blocks covering initial state, `startRecording()`, `stopRecording()`, `clearRecording()`, the duration timer, and the WAV conversion fallback.

Table 5.8: Voice Recorder Hook Test Distribution

Group	Success	Error/Edge	Total
Initial state	1	0	1
<code>startRecording()</code>	3	2	5
<code>stopRecording()</code>	2	1	3
<code>clearRecording()</code>	0	2	2
Duration timer	0	3	3
WAV conversion fallback	0	1	1
Total	6	9	15

Microphone permission denial (test VR-05). `getUserMedia` is mocked to reject with a `NotAllowedError`. The test asserts that `isRecording` remains `false` and the error message matches `/microphone access denied/i`. A separate test (VR-06) covers the generic hardware failure case.

Maximum duration auto-stop (test VR-13). The hook is instantiated with `maxDuration = 5000` ms. After advancing fake timers by 6000 ms, the test asserts that `isRecording` has been automatically set to `false`. This verifies the safety cap that prevents indefinite recording sessions on mobile devices.

WAV conversion fallback (test VR-15). `global.AudioContext` is replaced with a class whose `decodeAudioData` returns `Promise.reject(...)`. The hook catches this and falls back to transmitting the raw recorded blob without WAV conversion. The test asserts that `audioBlob` is still a valid `Blob` instance, demonstrating that a corrupted or unsupported audio source does not block the user from completing the screening.

5.5.3 Image Validation Tests (`imageValidation.test.js`)

These 23 tests cover four pure functions exported from `imageValidation.js` in the client utilities module. No React rendering or API calls are involved.

Table 5.9: Image Validation Test Distribution

Function	Success	Error/Edge	Total
<code>validateImageFile(file)</code>	3	5	8
<code>validateImageDimensions(file)</code>	2	4	6
<code>getConfidenceLevel(score)</code>	3	2	5
<code>getConfidenceColor(score)</code>	3	1	4
Total	11	12	23

Boundary conditions verified. `validateImageFile` accepts files at exactly the 10 MB limit (test IV-03) but rejects files at 10 MB+1 byte (test IV-06). `validateImageDimensions` accepts an image at exactly 300×300 pixels (test ID-02) but rejects 299×300 on the width axis (test ID-06). `getConfidenceLevel` treats both boundaries as inclusive: 0.8 is high, 0.6 is medium, anything below 0.6 is low.

Error accumulation (test IV-07). A file that is both the wrong type and oversized produces two error messages, not one. The test passes a 20 MB GIF and asserts that `errors.length === 2`. This contract is important for the registration form, which displays all violations at once instead of forcing the user to fix and resubmit repeatedly.

5.6 Inference (Python) Tests

The inference service test suite is located in `inference/tests/` and covers four areas: the health and prediction endpoints, the transcription endpoint, heatmap file serving, and unit tests for utility functions imported directly from `server.py`.

5.6.1 Prediction and Health Tests (test_predict.py)

This file contains 19 tests, of which 5 cover `GET /health` and 14 cover `POST /predict`. Health tests verify that the response body advertises the seven dermatological conditions, the device, the model version string, and the ASR loading state. Prediction tests verify response shape, numeric contracts, file output, and rejection of malformed inputs.

Table 5.10: Numeric and Shape Contracts Verified for `POST /predict`

Field	Contract Verified
<code>top_condition</code>	Is one of the seven valid class names.
<code>confidence_score</code>	Is in $[0.0, 1.0]$.
<code>all_scores</code>	Has all seven class names as keys.
<code>all_scores sum</code>	Sums to approximately 1.0 within $\epsilon = 0.01$ (verifies softmax).
<code>heatmap_path</code>	Ends with <code>.png</code> ; file exists at <code>HEATMAP_DIR/filename</code> .
<code>inference_time_ms</code>	Is a positive integer.
<code>model_version</code>	Is a non-empty string.

Malformed input rejection. Non-image bytes (test PR-11), an empty payload (PR-12), and a truncated JPEG header containing only the first three bytes `0xFF 0xD8 0xFF` (PR-13) all return 400. Test PR-14 confirms that a valid 2048×2048 image is accepted: the preprocessing pipeline resizes it to 224×224 before inference, so large source images do not cause memory or timeout errors.

5.6.2 Transcription Tests (test_transcribe.py)

Because `ASR_MODEL_PATH` is nonexistent, `asr_pipe` is `None` at startup. Every `/transcribe` request returns 503. The 9 tests in this file verify that the 503 contract is correctly implemented and that responses remain well-formed JSON across input variations: WAV (TR-01), WebM (TR-04), MP4 (TR-05), 255-character filenames (TR-08), and the `None` filename edge case (TR-09, where status may be 422 if FastAPI’s validator fires before the handler). Test TR-06 covers the empty-payload case where either the explicit empty-file guard or the ASR guard may fire first; the test asserts `status_code` in $(400, 503)$, accepting either valid resolution.

5.6.3 Heatmap Endpoint Tests (test_heatmap.py)

These 10 tests first call `POST /predict` to generate a real heatmap PNG file in `HEATMAP_DIR`, then retrieve that file via `GET /heatmaps/{filename}`.

Table 5.11: Heatmap Endpoint Test Distribution

Category	Count	Coverage
Success	4	Status, Content-Type, body length, PNG signature
Error	2	Nonexistent filename, random UUID
Security	2	Path traversal: <code>../../../../etc/passwd</code> and <code>%2Fetc%2Fpasswd</code>
Edge	2	Missing <code>.png</code> extension, empty filename segment
Total	10	

PNG signature verification (test HM-04). A successful response is asserted to begin with the eight-byte PNG signature `0x89 0x50 0x4E 0x47 0x0D 0x0A 0x1A 0x0A`. This is a stronger contract than checking the `Content-Type` header alone: it confirms that the file content actually decodes as a PNG image, not that the server has merely labelled it as one.

Path traversal defence (tests HM-07, HM-08). The `GET /heatmaps/{filename}` endpoint joins the user-supplied filename with `HEATMAP_DIR` using `os.path.join`. Without sanitisation, a payload like `../../../../etc/passwd` could read files outside the designated directory. The endpoint rejects any filename that resolves outside `HEATMAP_DIR`. Both `../` traversal (HM-07) and URL-encoded path separators `%2F` (HM-08) are tested. The assertion accepts any of 400, 404, or 422: the server must never return 200 for a path that escapes the sandbox.

5.6.4 Utility Function Tests (test_utils.py)

These 17 tests import utility functions directly from `server.py` and verify their behaviour without HTTP overhead.

Table 5.12: Utility Function Test Distribution

Function	Verified Behaviour	Tests
<code>load_audio_wav(path)</code>	Returns float32 array at 16 kHz; correct sample count for 1 s, 0.1 s, and 0.2 s clips; silent input is all-zero; missing file raises.	6
GradCAM class	CAM is 2-D; values in [0.0, 1.0]; <code>class_idx</code> is in valid range; explicit <code>class_idx</code> is honoured.	4
<code>create_heatmap_overlay()</code>	Output is uint8 3-channel array at <code>IMG_SIZE</code> resolution; pixel values in [0, 255]; degenerate all-zero and all-one CAMs do not produce NaN.	7
Total		17

Numerical stability under degenerate inputs (tests HO-06, HO-07). The overlay function normalises the CAM to $[0, 1]$ before applying the jet colourmap. If the CAM is uniform (all zeros or all ones), naive min-max normalisation would divide by zero and produce NaN values. Tests HO-06 and HO-07 pass an all-zero 7×7 CAM and an all-ones 7×7 CAM respectively, and assert that the output remains a valid `uint8` array with no NaN. This guards against a class of failure where a degenerate model output silently corrupts the visualisation.

5.7 Key Design Decisions in the Test Suite

5.7.1 Graceful Fallback Tested via Natural Failure

The inference and ASR services are not mocked in the server integration tests. Instead, the tests rely on the natural failure that occurs when no service is listening on port 5001 (`ECONNREFUSED`). The controllers catch these errors and activate fallback behaviour: the voice controller stores a `null` transcript with `asr_available: false`; the inference controller generates a deterministic mock prediction. This design has two advantages over explicit mocking. First, it tests the actual error type that production will encounter when the Python service is restarting or crashed, rather than a fabricated error chosen by the test author. Second, it removes one layer of indirection between test and code under test, so a regression in the catch block surfaces as a test failure rather than a passing test against an out-of-date mock.

5.7.2 Random-Weight Inference Testing

The Python inference tests deliberately avoid the trained model weights. `MODEL_PATH` is pointed at a nonexistent file, so PyTorch loads the MobileNetV3-Large architecture with random weights. Predictions are meaningless, but every contract that does not depend on prediction quality is verifiable: the response shape, the softmax sum, the heatmap PNG signature, the GradCAM normalisation range, and the file-system side effects. This decouples the test suite from the trained weights file, which is large, version-controlled separately, and changes with each retraining cycle. Testing the algorithm with random weights also stress-tests the normalisation logic: if GradCAM divided by zero when `cam.max() == cam.min()`, test GC-02 would fail, even though the failure has nothing to do with the trained network.

5.7.3 Path Traversal Security

Tests HM-07 and HM-08 verify that the heatmap endpoint cannot be used to read arbitrary files from the server filesystem. The `../../../../etc/passwd` and `%2Fetc%2Fpasswd` payloads represent the two most common path traversal techniques: literal relative paths and URL-encoded path separators. The server must return 400, 404, or 422 for any filename that resolves outside `HEATMAP_DIR`, and never 200.

5.7.4 Selective Fake Timer Scope

The duration timer tests in `useVoiceRecorder.test.jsx` use a restricted fake-timer scope, configured as

```
vi.useFakeTimers({ toFake:
  ['setInterval', 'clearInterval', 'Date'] }),
```

rather than the unrestricted `vi.useFakeTimers()`. The full fake timer mode also replaces `setTimeout`, which React 19 uses internally for scheduling concurrent updates. Replacing `setTimeout` causes `renderHook` to return `null` for `result.current` because React's scheduler is unable to flush state updates. The selective approach fakes only the APIs that the hook itself uses for its duration counter, leaving React's scheduler undisturbed.

5.7.5 Two-Layer Validation

User-supplied input is validated at two independent layers wherever an attacker could bypass the client. The age field in registration is checked in `RegisterPage.jsx` for non-integer, negative, zero, or over-120 values, and again in `authController.js` before any database query. The image upload endpoint validates MIME type and file size in the browser via `validateImageFile`, and again in the multer filter on the server. The

server-side check is the security boundary; the client-side check exists for user experience only.

5.8 Incorrect Examples Handled

- **Authentication and security.** Malformed JWTs, tampered tokens, expired tokens, blacklisted (logged-out) tokens, missing `Authorization` headers, and `Basic` scheme tokens are all rejected with strict 401 responses. Role mismatch on protected routes returns 403; an unauthenticated request to a role-protected route returns 401, because authentication runs before authorisation.
- **Media validation.** Non-image MIME types submitted to image endpoints (PDF, GIF) and non-audio MIME types submitted to audio endpoints (PNG, plain text) are rejected by the multer filter. Files exceeding 10 MB or below the 300×300 pixel minimum are rejected client-side before the request is sent.
- **Identifier confidentiality.** The login endpoint returns the same ‘`Invalid credentials`’ string for both “user not found” and “wrong password” cases, preventing username enumeration through differential responses.
- **Inference inputs.** Truncated JPEG headers, empty byte payloads, and arbitrary text payloads submitted to `/predict` are rejected with 400 before the model is invoked.
- **Path traversal.** Filenames containing `../` sequences or URL-encoded separators are rejected by the heatmap endpoint, preventing arbitrary file reads outside `HEATMAP_DIR`.

5.9 Edge Cases Handled

- **ASR service offline.** When the Python ASR service is unreachable (`ECONNREFUSED`), the screening flow continues, the transcript is stored as `null`, and the response sets `asr_available: false`. The patient is not blocked from completing the screening.
- **Inference service unreachable.** If the inference service is unreachable or the image file path no longer exists on disk, the controller stores a deterministic mock prediction and the screening completes with a clearly-marked mock model version, rather than crashing.
- **WAV conversion failure.** If `AudioContext.decodeAudioData` fails during WAV conversion, the system falls back to transmitting the raw recorded blob. No data is

lost.

- **GradCAM numerical stability.** The overlay function was tested against degenerate all-zero and all-one CAM inputs to verify that normalisation does not divide by zero and the colourmap produces no NaN values.
- **Boundary input sizes.** A 2048×2048 image is accepted by `/predict` (resized internally to 224×224); a 10 MB file is accepted by `validateImageFile` but a 10 MB + 1 byte file is rejected; ages of 1 and 120 are accepted but 0 and 121 are rejected.
- **Browser audio codecs.** Chrome’s `audio/webm;codecs=opus` and Firefox’s `audio/ogg` are accepted alongside `audio/wav` via the `startsWith('audio/')` filter and the codec-suffix-stripping extension picker.
- **Asynchronous cleanup.** “Force exit” scenarios in Jest are managed so the test runner closes cleanly without orphaned database connections or hanging timers.

5.10 ML Evaluation

The AI model was evaluated on a held-out test set of 2,438 images unseen during training or validation. Per-class results are reported in Chapter 4. Seborrheic Dermatitis remains the weakest performer ($F1 = 0.39$) due to fundamental data scarcity: only 171 training images across all five datasets.

Bibliography

- [1] Pakistan Telecommunication Authority. (2024). *Annual Telecom Indicators*. Islamabad: PTA.
- [2] Pakistan Bureau of Statistics. (2023). *Pakistan Social and Living Standards Measurement Survey*. Islamabad: PBS.
- [3] Topol, E. J. (2019). *Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again*. Basic Books, New York.
- [4] Adesanya, T., et al. (2021). Using a machine learning progressive web application to predict child mortality risk in the Gambia. *Frontiers in Public Health*, 9.
- [5] Bilal, A., Rextin, A., & Ali, A. (2017). Roman-txt: Forms and functions of roman Urdu texting. In *Proceedings of the 19th International Conference on Human-Computer Interaction with Mobile Devices and Services* (pp. 1–9).
- [6] Zaidi, S., et al. (2020). Operability, usefulness, and task-technology fit of an mHealth app for delivering primary health care services by community health workers in underserved areas of Pakistan and Afghanistan. *Journal of Medical Internet Research*, 22(9), e18414.
- [7] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- [8] Selvaraju, R. R., et al. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of ICCV* (pp. 618–626).